

Watchdog Timer

Watchdog Timer

- 1. Watchdog Overview
- 2. Core Concepts
 - 2.1 How the Watchdog Works
 - 2.2 The Three-Phase Logic of This Experiment
 - 2.3 Core API
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Details
 - 5.1 Parameter Configuration and Initialization
 - 5.2 Phase 1 — Auto-Feed
 - 5.3 Phase 2 — Button Feed + Countdown
 - 5.4 Phase 3 — Timeout Reset
- 6. Operation Procedure
 - 6.1 Flashing and Running
 - 6.2 Normal Phenomenon
 - 6.3 Verification Methods
- 7. Common Problems

1. Watchdog Overview

Watchdog is an independently running hardware countdown timer. When the system is normal, the software periodically "feeds the dog" (refreshes the counter); if the program crashes, enters an infinite loop, or deadlocks and fails to feed the dog, the counter reaches zero → the hardware automatically resets the MCU, achieving "unattended self-healing".

This experiment demonstrates the full lifecycle of the watchdog in three phases:

- 1. **Auto-feed** — the program automatically feeds 5 times, proving that normal feeding does not cause a reset
- 2. **Manual feed + countdown** — stop auto-feeding and feed the dog manually via the BOOT button; each button press resets the countdown
- 3. **Timeout reset** — no button press within 5s → countdown reaches zero → stop feeding → the hardware WDT automatically resets the system

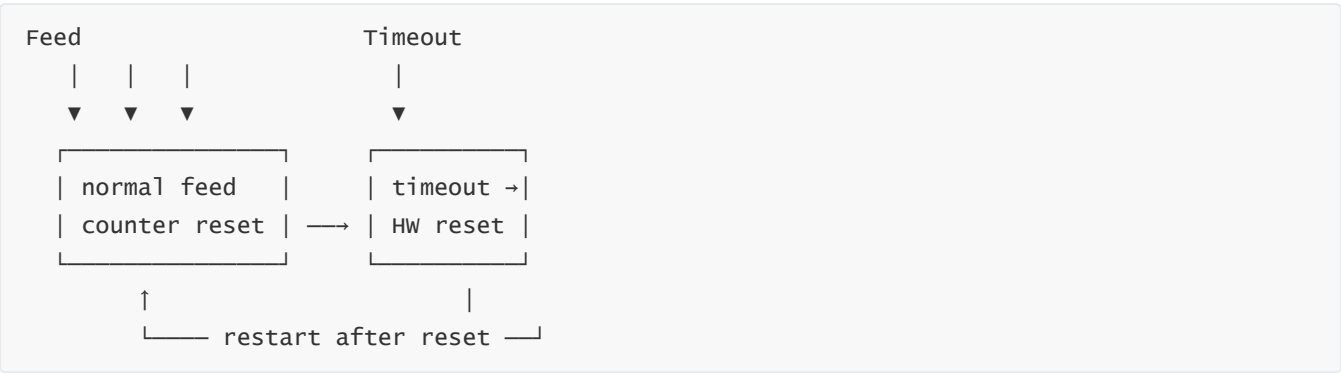
Typical application scenarios:

Application Type	Example
Anti-crash protection for products	Essential for mass-produced devices to ensure self-recovery when unattended
Network communication	Automatically restart when WiFi/BLE connection hangs
Sensor reading	Reset on peripheral timeout/no response

Application Type	Example
Low-power wake-up	Enter the normal initialization flow after a watchdog reset

2. Core Concepts

2.1 How the Watchdog Works



Each call to `wdt.feed()` resets the WDT counter to the `timeout` value. If the dog is not fed again before the counter reaches zero → the WDT hardware automatically resets the chip.

2.2 The Three-Phase Logic of This Experiment

Phase 1: Auto-feed	Phase 2: Button feed + countdown	Phase 3: Timeout reset
<pre>for i in 1..5: wdt.feed() print("Feed #") sleep(1000ms)</pre>	<pre>while remaining > 0: if BOOT falling edge: wdt.feed() remaining = 5 if 1s elapsed: remaining -= 1 print("Countdown") sleep(50ms)</pre>	<pre>while True: pass</pre>

2.3 Core API

```
from machine import WDT

wdt = WDT(timeout=7000) # enable WDT, 7s timeout
wdt.feed()              # feed: reset the counter to timeout
```

3. Hardware Dependencies

GPIO	Peripheral	Mode	Description
0	BOOT button	<code>Pin.IN</code> + <code>Pin.PULL_UP</code>	Trigger source for manual feeding in Phase 2

GPIO	Peripheral	Mode	Description
—	WDT	Hardware timer	Independent of the CPU, 7s timeout

4. Project Architecture and Flow

```
Script execution (single-file .py)
|
|─ Parameter config: FEED_COUNT=5, COUNTDOWN_SEC=5, POLL=50ms
|─ boot = Pin(0, IN, PULL_UP)           // BOOT button
|─ wdt = WDT(timeout=7000)              // WDT 7s timeout
|
|─ [Phase 1] for i in 1..5:              // auto-feed 5 times
|   |─ wdt.feed()
|   |─ print("Feed #d")
|   |─ sleep_ms(1000)
|
|─ [Phase 2] while remaining > 0:        // button feed + countdown
|   |─ pressed = boot.value() == 0      // read button
|   |─ if falling edge (pressed && !last): // on the press moment
|       |─ wdt.feed()                  // feed
|       |─ remaining = COUNTDOWN_SEC   // reset countdown
|       |─ tick = 0                    // reset tick
|   |─ last_state = pressed             // update state
|   |─ if tick % 20 == 0: print("Countdown") // print every second
|   |─ sleep_ms(50)                    // 50ms poll
|   |─ tick++; if tick >= 20: tick=0, remaining--
|
|─ [Phase 3] print("Watchdog timeout!")  // timeout reset
|   |─ while True: pass                 // stop feeding, wait for WDT HW reset
```

5. Source Code Details

Full source code: [5.Source Code](#) → [MicroPython](#) → [1.Basic Peripherals](#) → [5.WATCH_DOG](#) → [5.WATCH_DOG.py](#)

5.1 Parameter Configuration and Initialization

```
from machine import WDT, Pin # WDT + GPIO
import time                  # delay

FEED_COUNT      = 5          # auto-feed count
FEED_INTERVAL_MS = 1000      # feed interval (ms)
COUNTDOWN_SEC  = 5          # countdown after stopping (seconds)
POLL_INTERVAL_MS = 50        # button poll interval (ms)
TICKS_PER_SEC    = 1000 // POLL_INTERVAL_MS # poll ticks per second

# initialize the BOOT button (GPIO0, pull-up input, press=LOW)
boot = Pin(0, Pin.IN, Pin.PULL_UP)
```

```
# enable the hardware watchdog, 7s timeout (2s margin beyond the 5s countdown)
wdt = WDT(timeout=7000)
```

`WDT(timeout=7000)` has a 7s timeout, 2s more than the 5s countdown, ensuring that the board is not reset prematurely before the countdown reaches 0. `TICKS_PER_SEC = 20` is derived from 1000ms / 50ms.

5.2 Phase 1 — Auto-Feed

```
# Phase 1 - Auto-feed (5 times, 1s interval)
for i in range(1, FEED_COUNT + 1):
    wdt.feed()
    print("Feed #%d" % i)
    time.sleep_ms(FEED_INTERVAL_MS)
```

The 5 auto-feeds take about 5s in total and demonstrate that the system does not reset during normal feeding.

5.3 Phase 2 — Button Feed + Countdown

```
# Phase 2 - Manual feed + countdown
# Poll BOOT every 50ms; falling edge → feed immediately, reset countdown
print("Auto-feeding stopped! Press BOOT to feed...")

remaining = COUNTDOWN_SEC
tick      = 0
last_state = False

while remaining > 0:
    pressed = (boot.value() == 0)    # LOW = pressed

    # Falling-edge detection - feed only on release→press, no repeat on hold
    if pressed and not last_state:
        wdt.feed()
        print(">>> Boot feed! WDT reset <<<")
        remaining = COUNTDOWN_SEC
        tick      = 0
        last_state = pressed

    if tick % TICKS_PER_SEC == 0:
        print("Countdown %d ..." % remaining)

    time.sleep_ms(POLL_INTERVAL_MS)

    tick += 1
    if tick >= TICKS_PER_SEC:
        tick = 0
        remaining -= 1
```

Every button press before the countdown reaches zero resets `remaining = 5` and `tick = 0`, which is equivalent to "extending life". Falling-edge detection ensures that holding the button does not feed repeatedly.

5.4 Phase 3 — Timeout Reset

```
# Phase 3 – Timeout reset
# Countdown reached 0, stop feeding, HW WDT will reset the system
print("Watchdog timeout! Waiting for WDT reset...")
while True:
    pass
```

After the countdown reaches zero, the program enters an infinite loop and stops feeding; the hardware WDT resets the chip after the timeout.

6. Operation Procedure

6.1 Flashing and Running

Thonny IDE steps: See the `1. Before Development` → `3. Thonny IDE` chapter.

6.2 Normal Phenomenon

Shell output:

```
=====
ESP32-S3 Watchdog Test (MicroPython)
=====
BOOT button (GPIO0) ready – press to feed WDT
WDT timeout: 5s
Feed #1
Feed #2
Feed #3
Feed #4
Feed #5
Auto-feeding stopped! Press BOOT to feed...
Countdown 5 ...
>>> Boot feed! WDT reset <<<          <<----feed
Countdown 5 ...
Countdown 4 ...
>>> Boot feed! WDT reset <<<          <<----feed
Countdown 5 ...
Countdown 4 ...
Countdown 3 ...
Countdown 2 ...
>>> Boot feed! WDT reset <<<          <<----feed
Countdown 5 ...
Countdown 4 ...
Countdown 3 ...
Countdown 2 ...
```

```
Countdown 1 ...  
watchdog timeout! waiting for WDT reset... <-----timeout restart
```

Timeout reset/restart:

```
=====
ESP32-S3 Watchdog Test (MicroPython)
=====
BOOT button (GPIO0) ready – press to feed WDT
WDT timeout: 5s
Feed #1
Feed #2
Feed #3
Feed #4
Feed #5
Auto-feeding stopped! Press BOOT to feed...
Countdown 5 ...
Countdown 4 ...
Countdown 3 ...
Countdown 2 ...
Countdown 1 ...
Watchdog timeout! Waiting for WDT reset...
```

Manual feeding:

```
=====
ESP32-S3 Watchdog Test (MicroPython)
=====
BOOT button (GPIO0) ready – press to feed WDT
WDT timeout: 5s
Feed #1
Feed #2
Feed #3
Feed #4
Feed #5
Auto-feeding stopped! Press BOOT to feed...
Countdown 5 ...
>>> Boot feed! WDT reset <<<
Countdown 5 ...
Countdown 4 ...
>>> Boot feed! WDT reset <<<
Countdown 5 ...
Countdown 4 ...
Countdown 3 ...
Countdown 2 ...
>>> Boot feed! WDT reset <<<
Countdown 5 ...
Countdown 4 ...
Countdown 3 ...
Countdown 2 ...
Countdown 1 ...
Watchdog timeout! Waiting for WDT reset...
```

6.3 Verification Methods

Operation	Expected Result
Power up without pressing any button	Phase 1 auto-feeds 5 times → Phase 2 countdown → resets after 5s

Operation	Expected Result
Press BOOT every 4s during Phase 2	The countdown keeps resetting to 5; never resets
Hold BOOT during Phase 2	Feeds only once at the press moment; holding does not feed repeatedly
Change <code>timeout=7000</code> to <code>timeout=4000</code>	Countdown shortened to 4s, timeout triggered faster
Comment out <code>wdt.feed()</code> in Phase 2	Even pressing the button does not feed; resets when the countdown reaches 0

7. Common Problems

Symptom	Cause	Solution
Resets before Phase 1 even finishes	<code>timeout=7000</code> too short, <code>FEED_INTERVAL_MS</code> too large	Ensure the feed interval < timeout
No response from the button in Phase 2	Falling-edge detection <code>not</code> <code>last_state</code> not effective	Confirm <code>last_state = pressed</code> at the end of the loop
Shell disconnects mid-print	Watchdog reset causes the USB serial port to disconnect	Normal phenomenon; Thonny reconnects automatically after the reset
<code>WDT(timeout=7000)</code> reports an error	Firmware version does not support WDT with arguments	Use <code>WDT()</code> without arguments for the default timeout
Countdown gets stuck	<code>tick</code> increment or reset logic is wrong	Confirm <code>tick = 0</code> and <code>remaining -= 1</code> after <code>tick >= TICKS_PER_SEC</code>